Kimi-K3 on AMD MI355X

From attention fundamentals to FP8 execution

A CONCEPTUAL FIELD GUIDE

Understand

Attention, KDA, MLA and MoE

Trace

One token across eight GPUs

Optimize

Precision, memory traffic and correctness

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

A learning path with six stops

Build each concept before using it to explain performance.

01

Attention from first principles

02

Kimi-K3 architecture

03

GPU execution and memory

04

Serving across eight GPUs

05

FP8 data flow and numerics

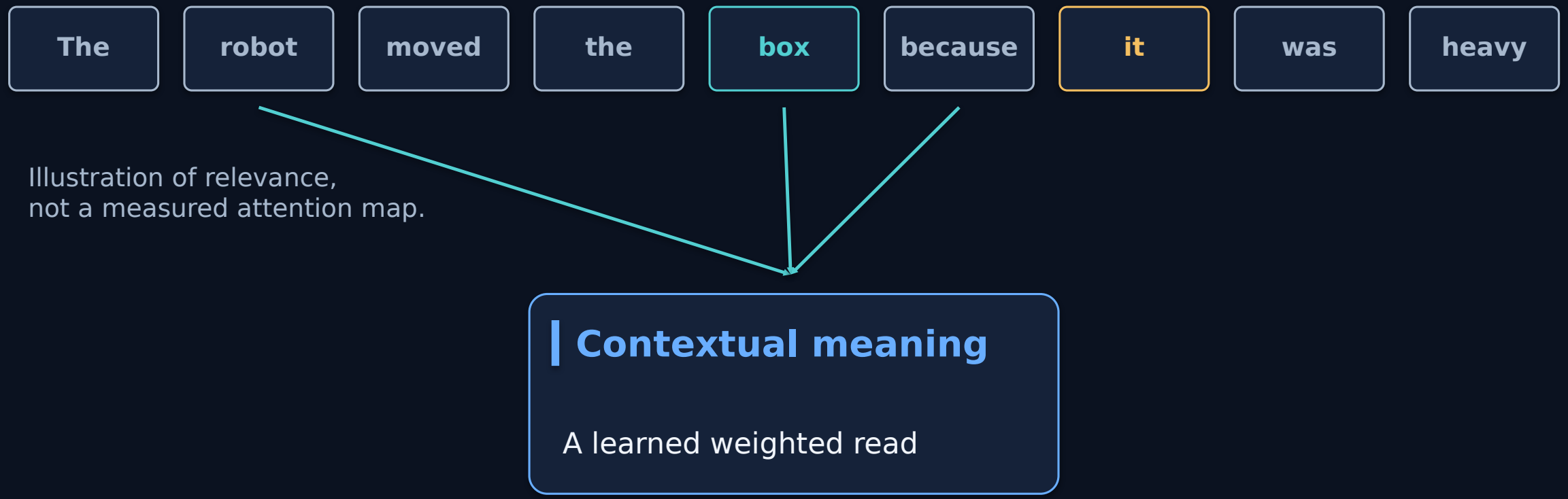
06

Evidence and optimization

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Attention asks: which earlier information matters?

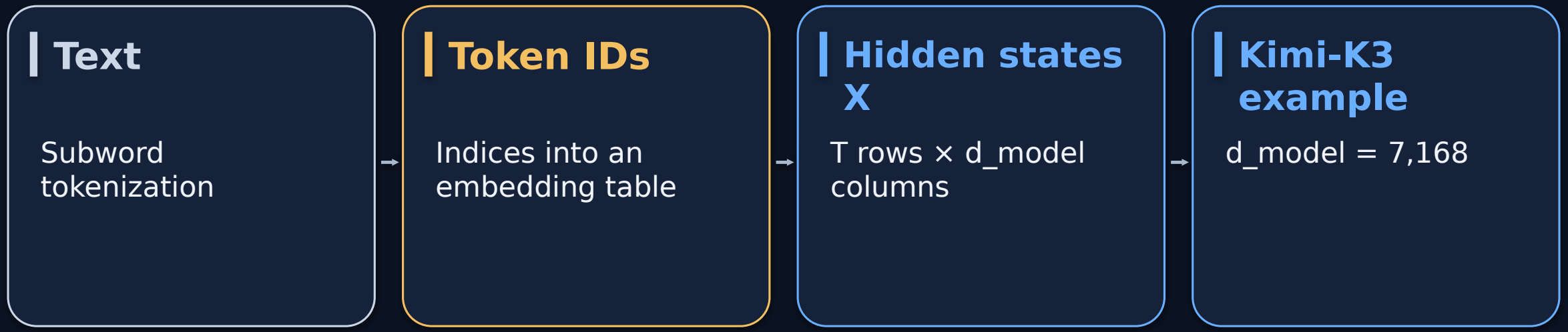
A token builds its next representation by reading relevant context.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Tokens become vectors; vectors become matrices

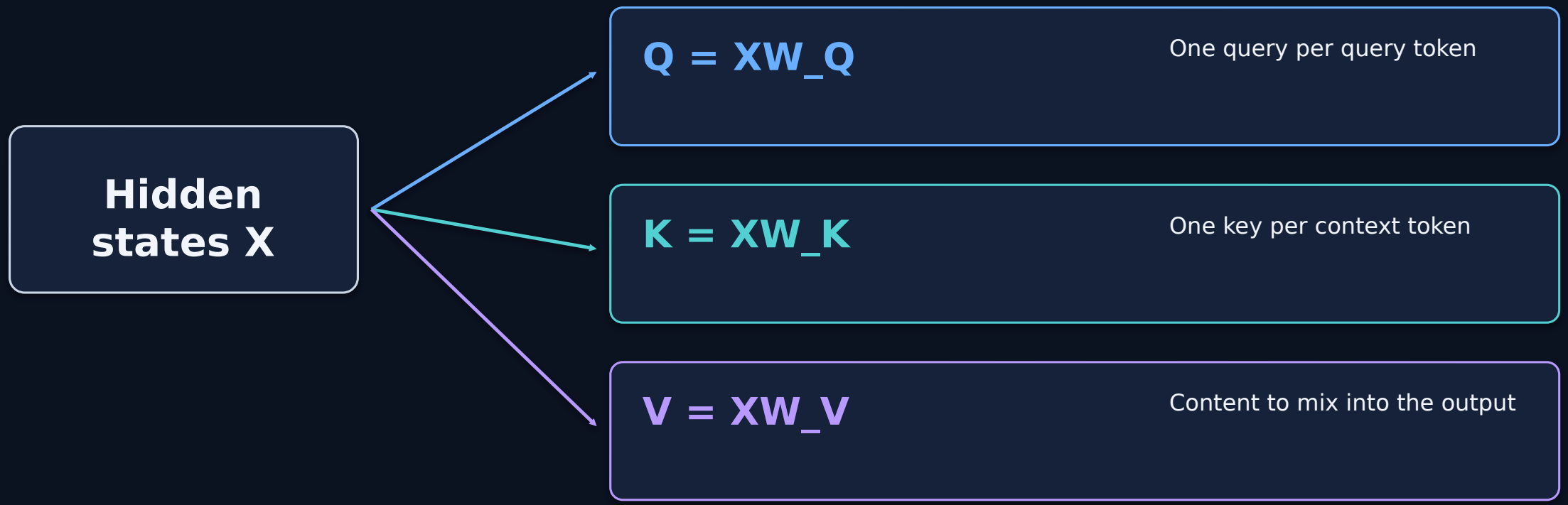
A token is an ID. Its hidden state is a learned list of numbers.



$$X \in R^{(T \times d_{model})}$$

Q, K and V are three learned projections

Query = what to look for. Key = how to match. Value = what to retrieve.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The dot product produces a matching score

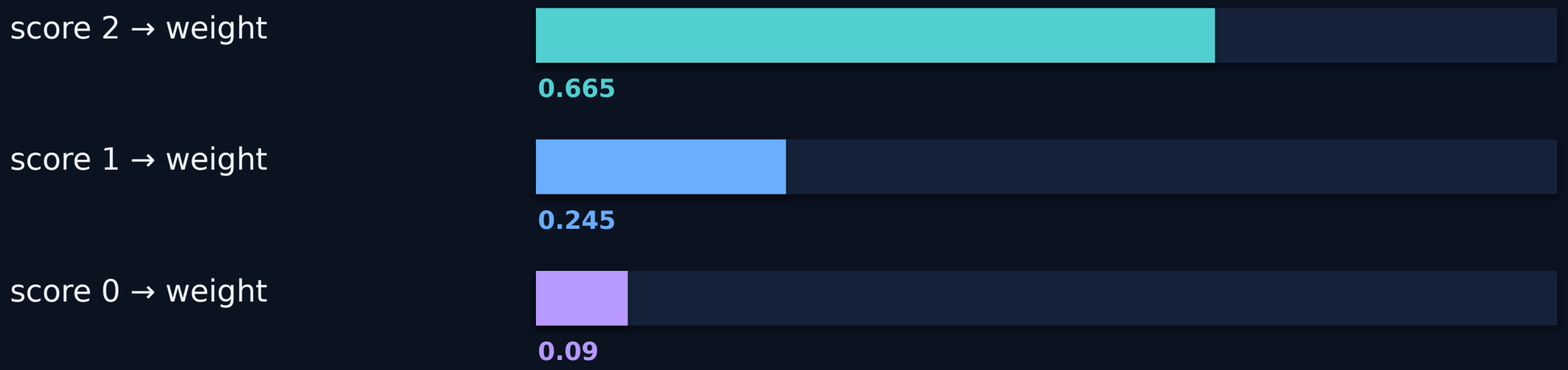
Aligned features reinforce each other; unrelated features contribute less.

1. **Multiply features** — $q = [1, 2],\ k = [3, 1]$
2. **Add the products** — $q \cdot k = 1 \times 3 + 2 \times 1 = 5$
3. **Control the scale** — Usually divide by $\sqrt{d_k}$

score(q, k) = (q · k) / √d_k

Softmax turns scores into relative weights

Subtract the maximum, exponentiate, and normalize each query row.



$$p_j = \exp(s_j - m) / \Sigma_i \exp(s_i - m), \quad m = \max_i s_i$$

The output is a weighted sum of values

Attention returns a vector—not the index of the best-matching token.

$V_1 = [10, 0]$

Weight 0.665

$V_2 = [0, 10]$

Weight 0.245

$V_3 = [5, 5]$

Weight 0.090

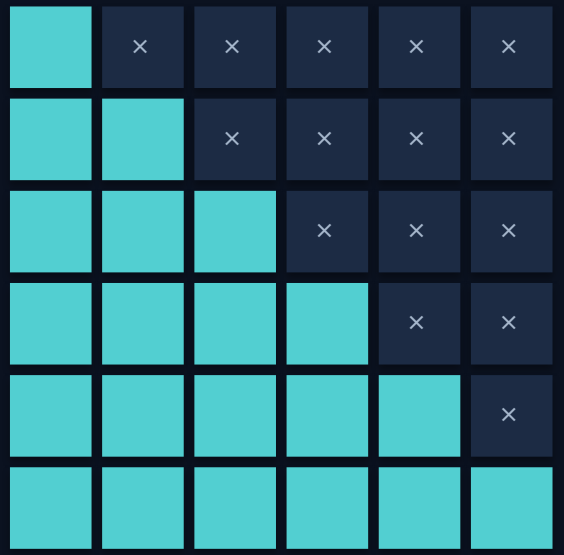
Sum the weighted vectors

$$O = 0.665V_1 + 0.245V_2 + 0.090V_3 \approx [7.10, 2.90]$$

Causal masking prevents future-token reads

A token can see the existing prefix and its own allowed position.

KEY POSITIONS →



QUERY ROWS ↓

Rows are queries

One row for each new token

Columns are keys

One column for each available token

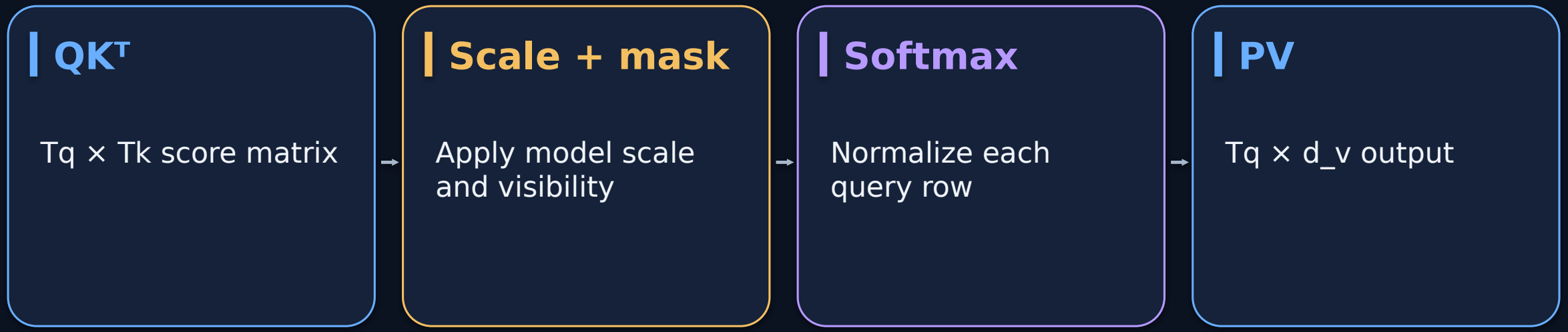
Future entries

Masked before softmax

M[i,j] = 0 for allowed keys; otherwise −∞

One equation joins the complete operation

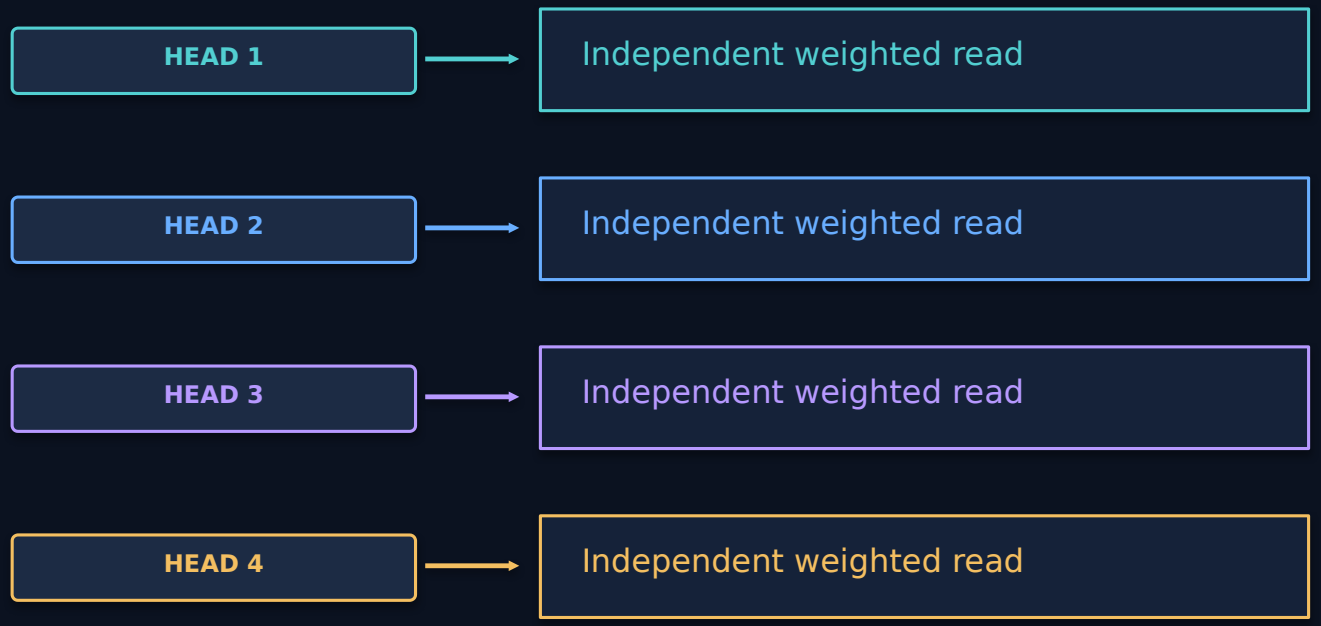
Two matrix products surround the softmax.



$$O = \text{softmax}(\alpha QK^T + M)V$$

Multiple heads learn multiple kinds of relationships

Each head produces its own weighted read; an output projection mixes them.



W_O

Mix head outputs into hidden features

$$Y = \text{concat}(O_1, \ldots, O_H)\, W_O$$

Residuals, normalization and MLPs complete the block

Attention is one part of a model layer.



$$\text{RMSNorm}(x) = g \odot x / \sqrt{(\text{mean}(x^2) + \varepsilon)}$$

Position information: separate the general idea from K3

RoPE is common in attention; this checkpoint's MLA uses NoPE.

General RoPE

Rotate query/key feature pairs according to position.

Kimi-K3 MLA

mla_use_nope = true; inspected wrapper passes rotary_emb=None.

Naming trap

A 64-column “RoPE” field still exists in the MLA layout.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Prefill and decode are different execution regimes

Prefill processes known tokens; decode produces the next token.

Prefill

Many query rows; large matrix products; new-token causal attention.

Decode

Few query rows per request; read cached history; repeat every output step.

Mixed serving

New prompt chunks can share a scheduler step with decoding requests.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The KV cache avoids recomputing the past

Cache persistent context; compute the new query for the current step.



Reuse + append

Keep the history consistent with the current request.

Standard KV bytes ≈ 2 × layers × tokens × KV_heads × head_dim × bytes/value

Why attention becomes expensive at long context

Every query can compare against many keys.

1. **Full prefill** — Attention pair count grows roughly with T^2
2. **One-token decode** — Attention pair count grows roughly with cached T
3. **Naive intermediates** — Scores and probabilities can dominate temporary memory

Attention FLOPs ≈ 2TqTk(d_k + d_v) per head

FlashAttention changes data movement

Tile the computation and keep temporary scores close to the compute units.



Load tile



Update m, l, a



Continue streaming

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Online softmax carries three pieces of state

A running maximum, a normalization sum, and a weighted accumulator.

m

Largest score seen so far

→

l

Rescaled sum of exponentials

→

a

Rescaled weighted value sum

→

Finish

O = a/l; LSE = m + ln(l)

m′ = max(m, max(s)); l′ = e^(m−m′)l + Σe^(s−m′)

A fused kernel can still lose end to end

Kernel speed must repay quantization, memory traffic and dispatch costs.

Kernel-only view

Time just the new FMHA call.

Execution-path view

Include preparation, quantization, layout changes and launches.

Serving view

Include scheduling, communication, cache behavior and errors.

Net saving = time removed − time introduced

Kimi-K3 is a hybrid model

Different mechanisms share the work of understanding long context.

KDA

69 layers: recurrent sequence mixing

Gated MLA

24 layers: compressed-cache attention

Latent MoE

Sparse expert computation

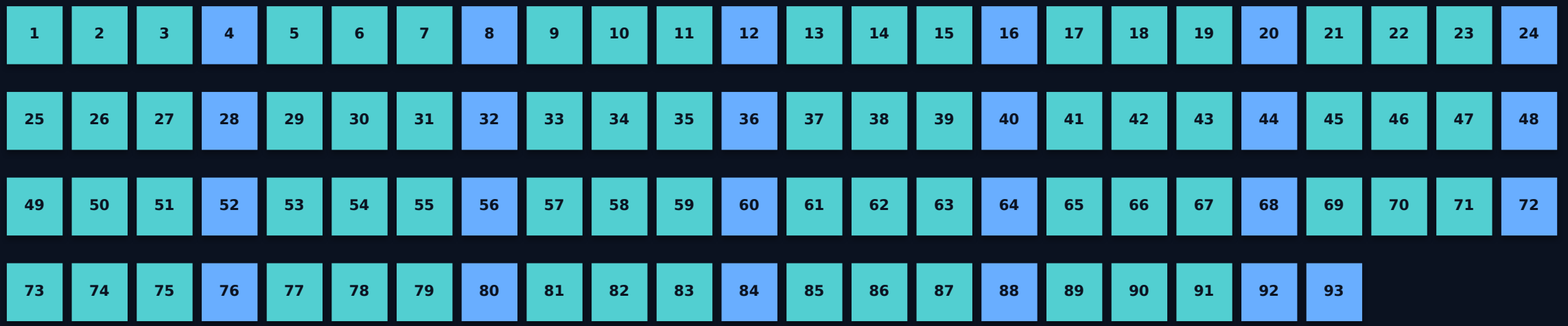
Attention Residuals

Learned mixing across model depth

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The actual 93-layer pattern

Mostly three KDA layers followed by one MLA layer, then a final MLA.



TEAL 69 KDA layers

BLUE 24 gated MLA layers

69 KDA + 24 MLA = 93 sequence-mixing layers

Checkpoint dimensions worth remembering

These numbers describe the referenced Kimi-K3 checkpoint.

7,168

Hidden-state width

96

Attention heads

512

Compressed KV latent width

1,536

Query latent width

896 / 16

Routed experts / selected per token

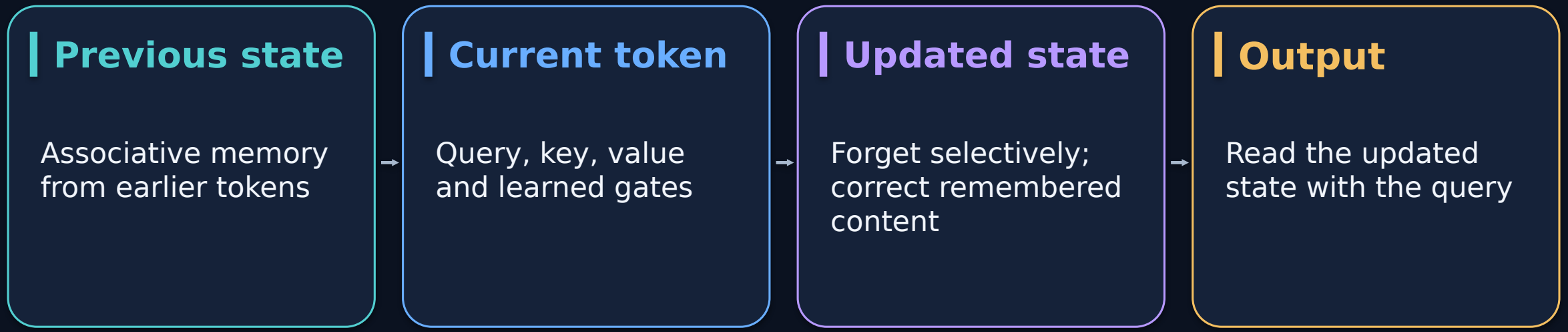
1,048,576

Configured maximum context

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

KDA keeps a learned summary of the past

Update a state matrix instead of storing a growing key/value list.



Persistent state passes to the next token →

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

KDA's update, one step at a time

Decay → predict → correct → read.

1 **Decay** $\bar{S} = \mathrm{diag}(\alpha)\, S_{\text{previous}}$

2 **Correct** $r = v - \bar{S}^{T}k; \quad S = \bar{S} + \beta k r^{T}$

3 **Read** $o = S^{T}q$

S ∈ R^(d_k × d_v); α controls forgetting; β controls update strength

KDA prefill and decode use different algorithms

The state is recurrent; processing a known chunk can still be parallelized.

Prefill

Chunked matrix operations process known token blocks.

Decode

Update a small persistent state for each arriving token.

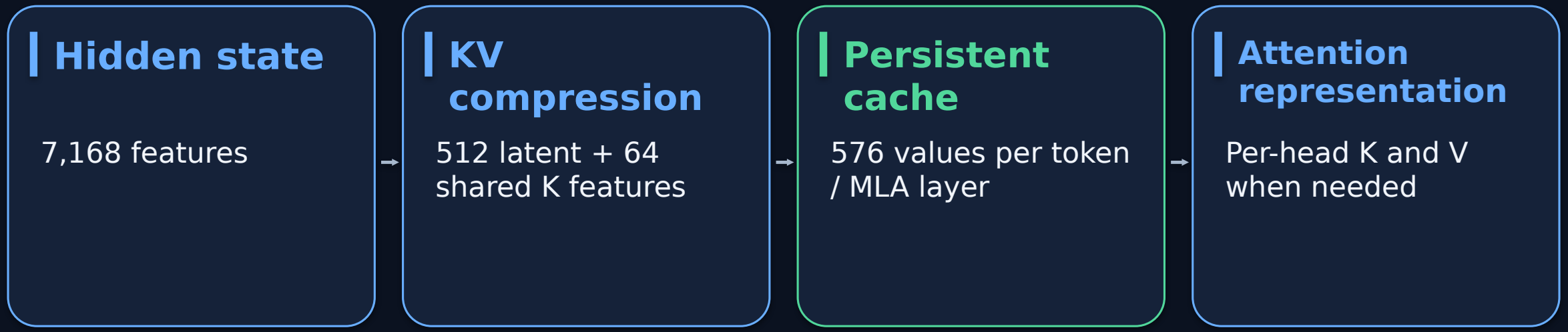
Speculative verification

Preserve state checkpoints and commit only accepted progress.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

MLA compresses the persistent token cache

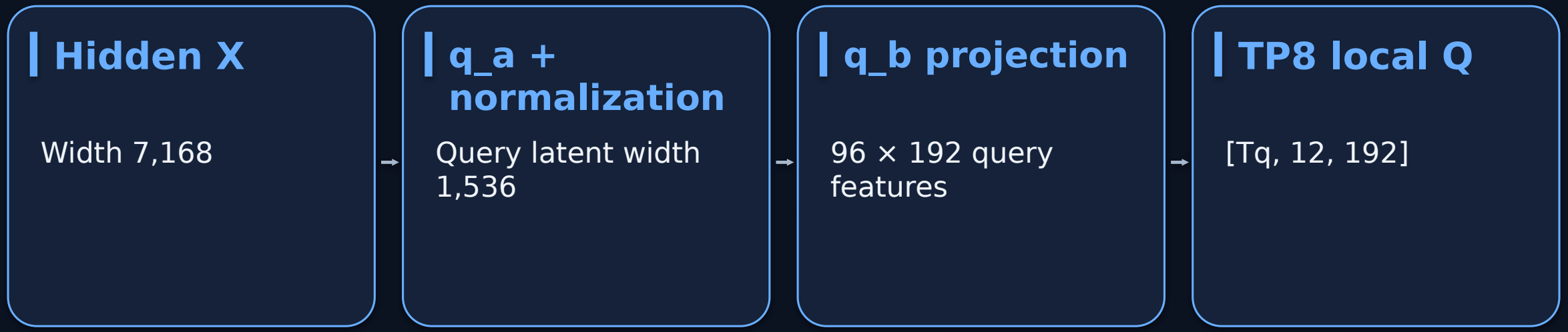
Store a shared latent per token, then reconstruct what attention needs.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Query preparation also uses a latent projection

The current token's query is expanded into per-head features.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

kv_b_proj reconstructs per-head keys and values

The weight matrix is neither the KV cache nor an FMHA input.



$Z = C\,W_{kv}^{T}$; 3,072 = 12 × (128 + 128)

MLA decode can avoid expanded K/V

Associativity lets projections move across the attention calculation.

Explicit prefill path

Build K/V per head, then call FMHA.

Latent decode path

Transform Q into latent space; attend to compressed cache.

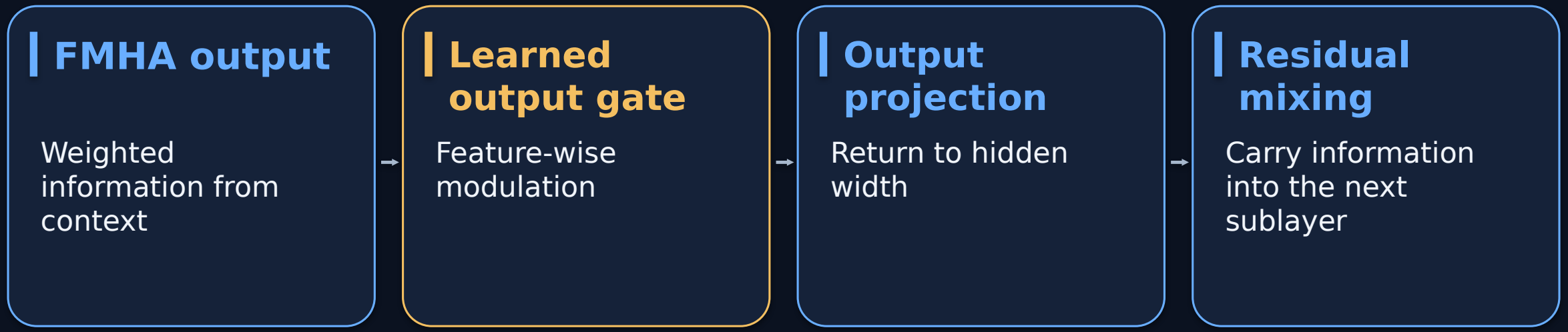
After the latent read

Apply the value projection to obtain each head's output.

$$q(CW_K^T)^T = (qW_K)C^T$$

Gated MLA controls the attention output

A learned gate modulates retrieved features before the next projection.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

MoE selects a small subset of a large expert pool

Each token activates 16 routed experts, plus the shared branch.

Router

One token
896 scores



Illustrated subset of the expert pool

Combine

16 selected
+ shared branch

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Latent MoE reduces the routed branch's width

Experts operate in a smaller feature space, then return to model width.



routed(x) = W_up · norm(Σ_e w_e · Expert_e(W_down x))

SiTU-GLU is the expert's gated nonlinearity

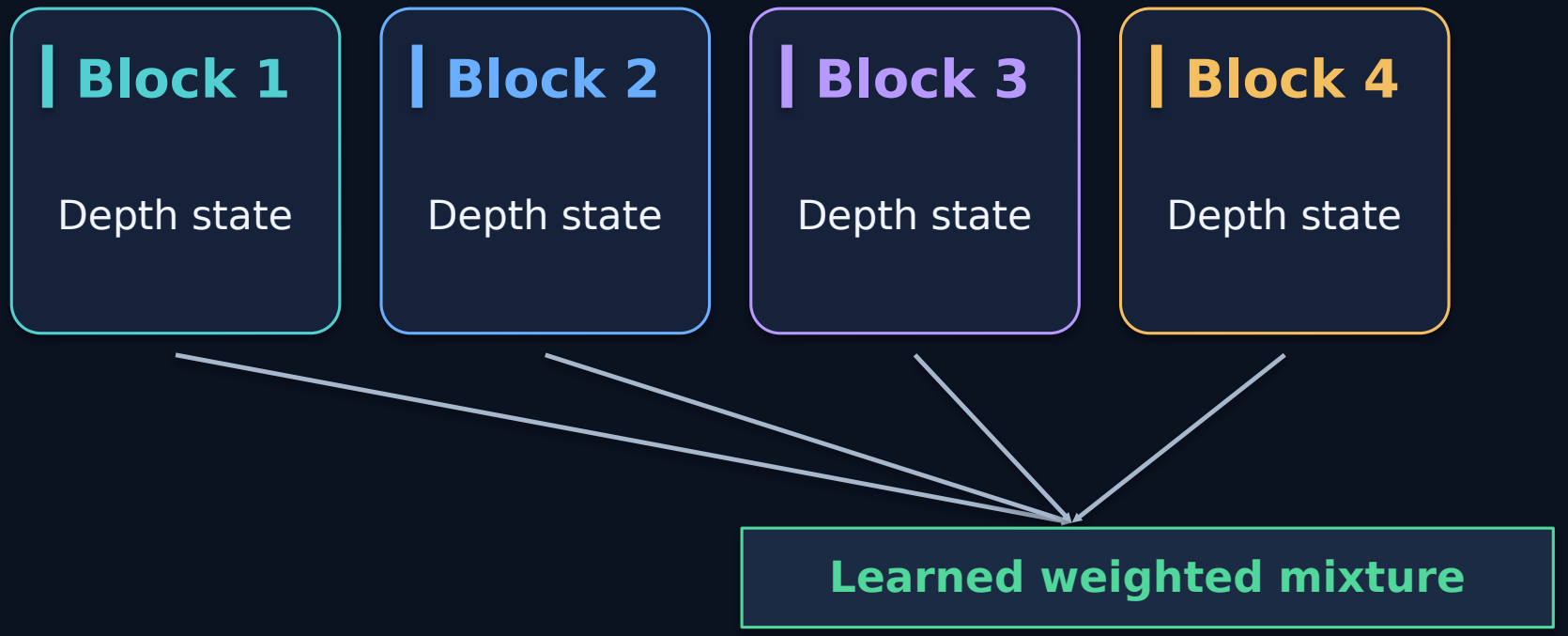
Two projection branches interact, with soft clipping to control range.

1. **Gate branch** — $g = 4 \tanh(a/4) \cdot \text{sigmoid}(a)$
2. **Value branch** — $u = 25 \tanh(b/25)$
3. **Multiply + project** — $z = g \odot u$; then apply the down-projection

Expert(x) = W_down [SiTU(W_gate x) ⊙ softclip(W_up x)]

Attention Residuals attend across depth

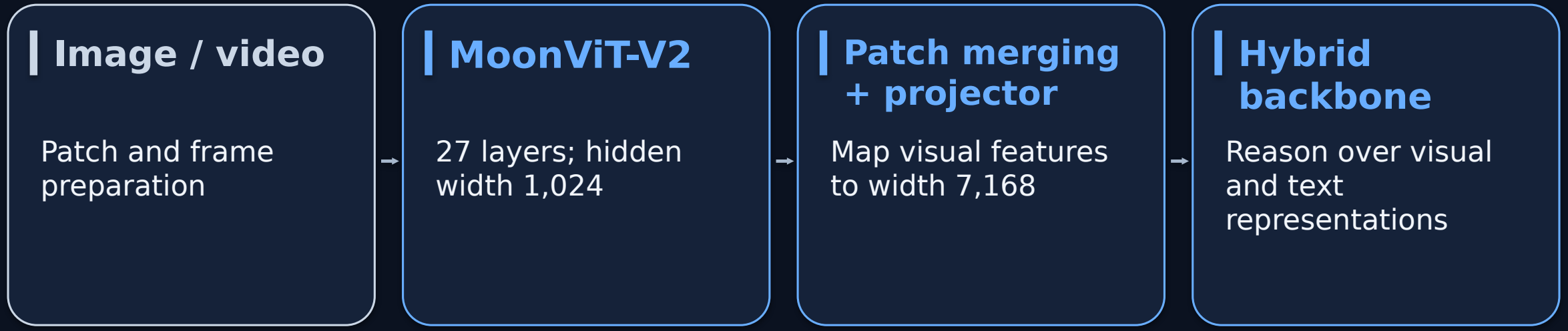
Learn which earlier layer-block representations should contribute.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Vision joins the model through visual tokens

A vision encoder and projector connect images/video to the text backbone.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

MI355X supplies compute, capacity and bandwidth

The GPU must move tensors and execute them efficiently.



HBM3E

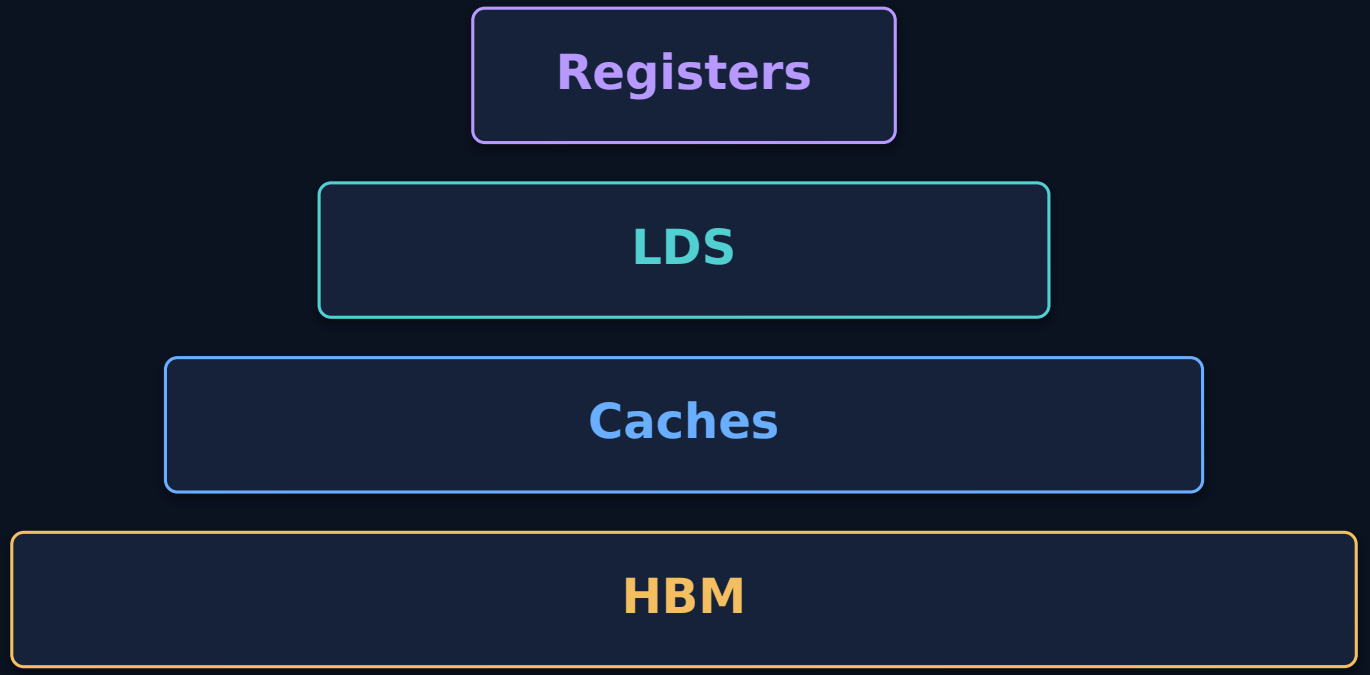
288 GB
advertised capacity

One GPU package

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The memory hierarchy determines reuse

Registers and LDS are small and close; HBM is large and farther away.



Per-thread / wave computation state

On-chip workgroup-shared tiles

Hardware-managed reuse

Weights, KV, activations and workspaces

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

A GEMM becomes tiles, waves and matrix instructions

Large matrix products reuse data across many multiply-accumulates.



A: tokens × features

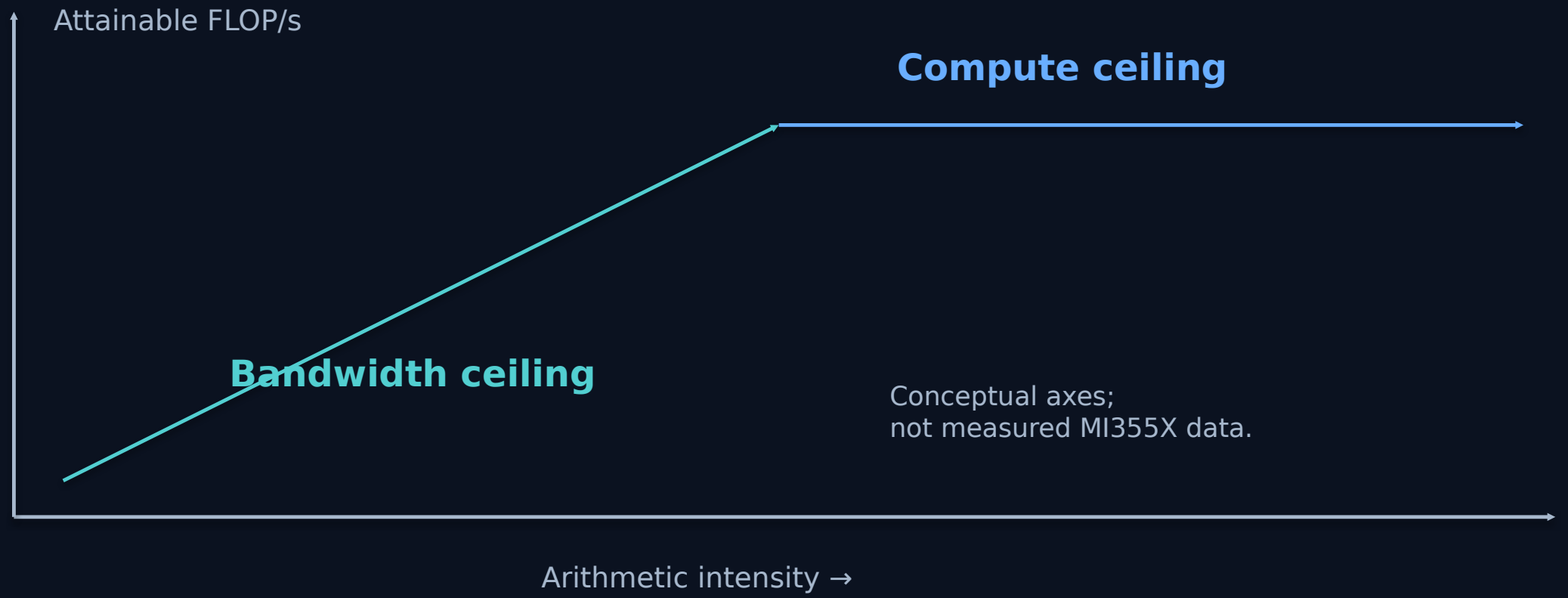
B: features × outputs

C: token outputs

C[M,N] = A[M,K] × B[K,N]; work ≈ 2MNK FLOPs

The roofline connects FLOPs to bytes

A kernel is limited by whichever resource it needs more urgently.



Attainable FLOP/s ≤ min(compute ceiling, bandwidth × intensity)

Why eight GPUs are needed

Sparse activation reduces work; the full weight set still consumes memory.

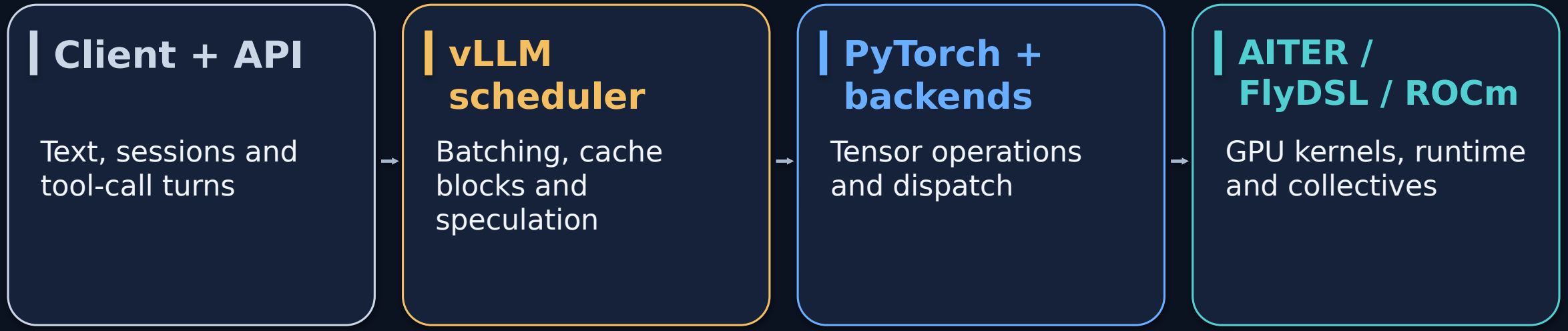


Scales, BF16 modules, caches and workspaces consume the remaining budget.

Ideal weight bytes = parameter count × bits per parameter / 8

The serving stack turns requests into kernels

Every layer of software has a different responsibility.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Tensor parallelism splits model calculations

TP8 divides selected weight dimensions across eight ranks.

GPU 0 Feature shard	**GPU 1** Feature shard	**GPU 2** Feature shard	**GPU 3** Feature shard
GPU 4 Feature shard	**GPU 5** Feature shard	**GPU 6** Feature shard	**GPU 7** Feature shard

96 heads / 8 TP ranks = 12 local prefill heads per rank

DCP partitions the context across the same ranks

TP8 and DCP8 use eight GPUs here—not sixty-four.

GPU 0	GPU 1	GPU 2	GPU 3
Context partition	Context partition	Context partition	Context partition

GPU 4	GPU 5	GPU 6	GPU 7
Context partition	Context partition	Context partition	Context partition

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Collectives move or combine distributed tensors

All-gather, all-reduce and all-to-all solve different problems.

All-gather

Everyone receives all participants' pieces.

All-reduce

Everyone receives a reduction, such as a sum.

All-to-all

Each participant sends a different piece to each peer.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Merge attention chunks with their normalization mass

Average by exp(LSE), not by chunk length or a fixed 50/50 split.

Chunk A

Output O_A and LSE_A

→

Chunk B

Output O_B and LSE_B

→

Stable mass weights

Normalize exponentials of the two LSEs

→

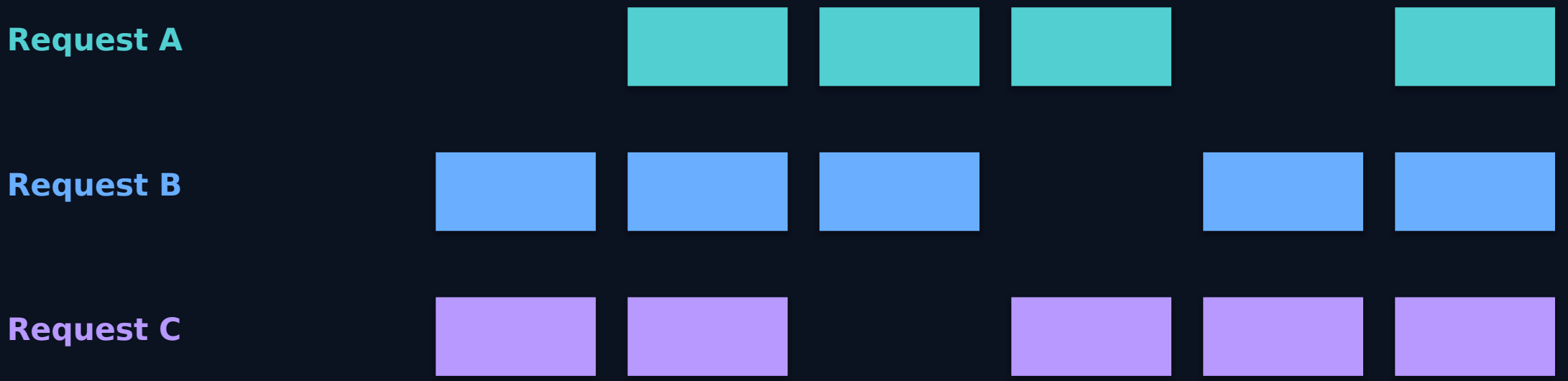
Merged output

w_A O_A + w_B O_B

L = logaddexp(L_A,L_B); O = e^(L_A−L)O_A + e^(L_B−L)O_B

Continuous batching keeps requests moving

The scheduler admits new work as other requests finish.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Prefix reuse and offload trade compute for movement

Agentic turns often repeat a long prefix and append a small suffix.



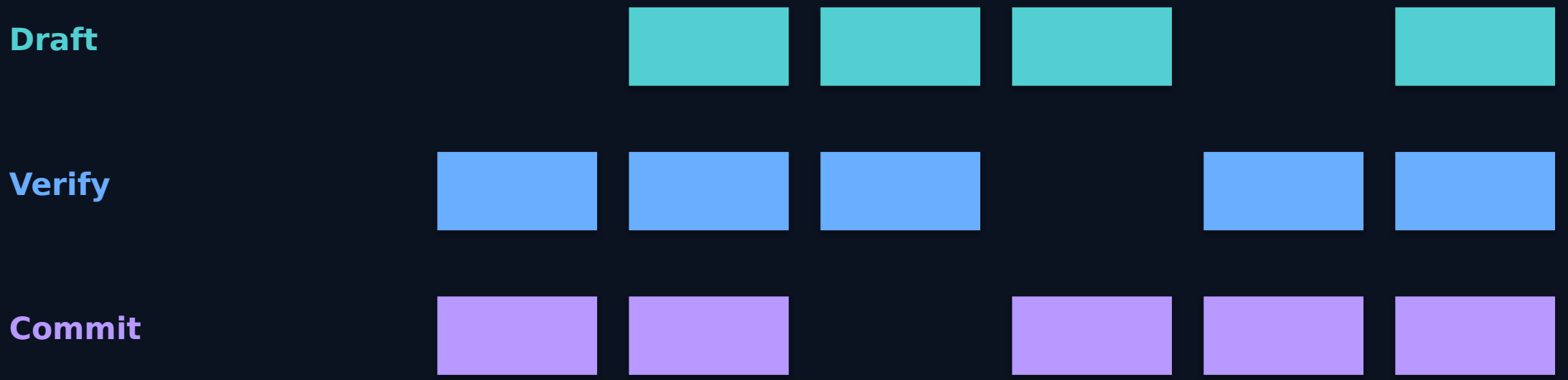
Reuse + append

Keep the history consistent with the current request.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

MTP proposes several tokens; the target verifies them

Speed depends on accepted progress per complete draft-and-verify cycle.



(schematic, not a measured trace)

Useful speedup ≈ accepted progress per cycle / relative cycle cost

CPU placement and GPU graphs address dispatch overhead

Locality and launch reuse matter when individual GPU operations are short.

NUMA / CCD binding

Place worker CPU threads near relevant memory and devices.

Graph replay

Reuse a captured GPU launch dependency graph.

Tradeoffs

Affinity needs topology; graph pools consume persistent memory.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Precision belongs to each object and each operation

“An FP8 path” does not mean every internal value uses eight bits.

MXFP4 weights

4-bit values with block scaling

FP8 operands

Compact Q/K/V and projection operands

BF16 activations

16-bit storage for many outputs

FP32 internals

Scales, reductions and accumulators

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Quantization changes numbers; dequantization rescales them

The scale preserves units, but it cannot recover rounding already lost.

Original X

Higher-precision values

Choose s

Use range / representable maximum

Quantize

$X_8 = \text{round_FP8}(\text{clip}(X/s))$

Interpret

$X \approx s \times X_8$

For E4M3FN: s = max|X| / 448, with explicit zero-range handling

Scale granularity changes the accuracy/performance tradeoff

Per-tensor, per-channel and per-block scales cover different value groups.

Per-tensor

One scale; simple metadata; outliers affect the whole tensor.

Per-channel

Separate output-channel weight scales; tighter ranges.

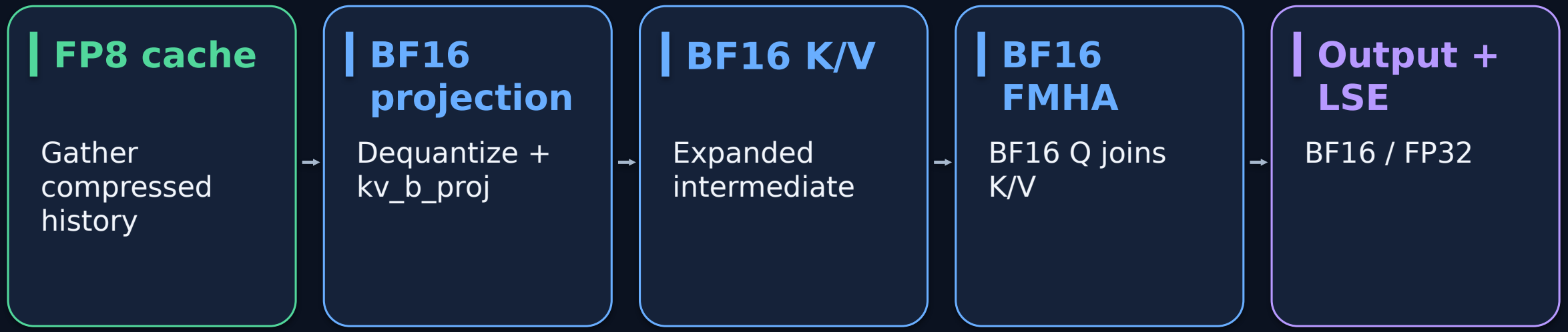
Per-block

Scales shared by small groups; extra packing and metadata.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Stage 0: before the FP8 attention optimizations

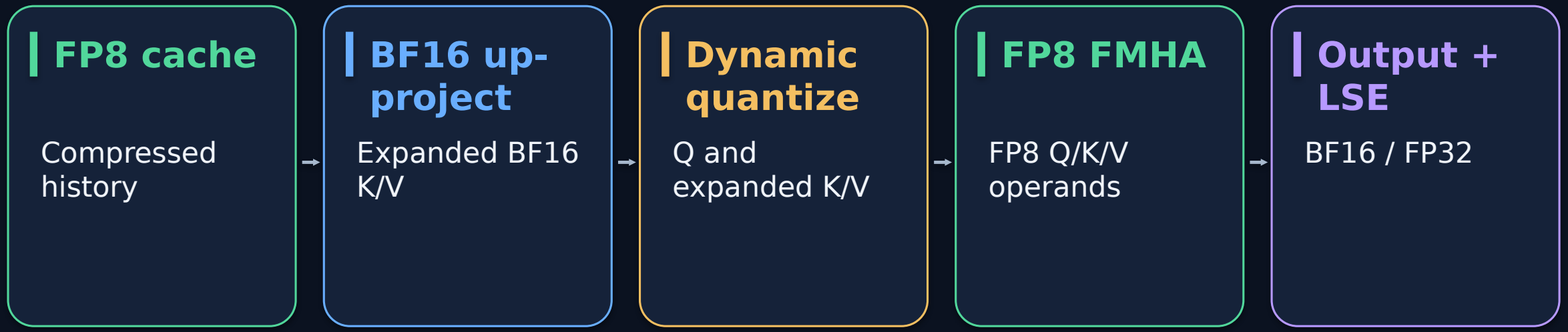
The compressed cache was already FP8; the attention operands were BF16.



Established full configuration: 15,476.46881 total tokens/s/GPU

Stage 1: dynamic operands for FP8 FMHA

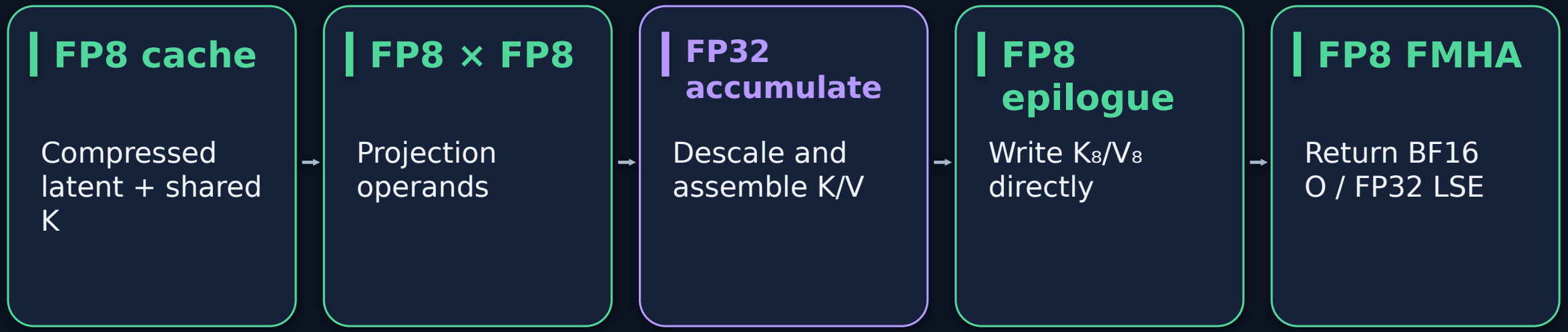
Keep BF16 up-projection, then quantize the expanded Q/K/V.



Established full configuration: 16,915.72154 total tokens/s/GPU

Stage 2: direct FP8 up-projection

FP8 cache × FP8 kv_b_proj → FP8 K/V storage → FP8 FMHA.



Established full configuration: 17,406.08912 total tokens/s/GPU

Follow the units through the FP8 projection

Input, weight and output scales play separate roles.

1. **Latent cache** — $C \approx s_C C_8$
2. **Weight channel j** — $W_j \approx s_{W,j} W_{8,j}$
3. **Accumulated projection** — $Z_j \approx s_C s_{W,j} \Sigma_i C_{8,i} W_{8,j,i}$
4. **FP8 output** — $K_8 = \text{round_FP8}(K/s_K)$; similarly for V

FP8 multiplication operands → FP32 accumulation → scaled FP8 epilogue

What direct output actually removes

It removes a large BF16 intermediate and its separate quantization pass.

Earlier path

Write BF16 K/V → read BF16 K/V → quantize → write FP8 K/V.

Direct path

Accumulate locally → scale/round → write FP8 K/V.

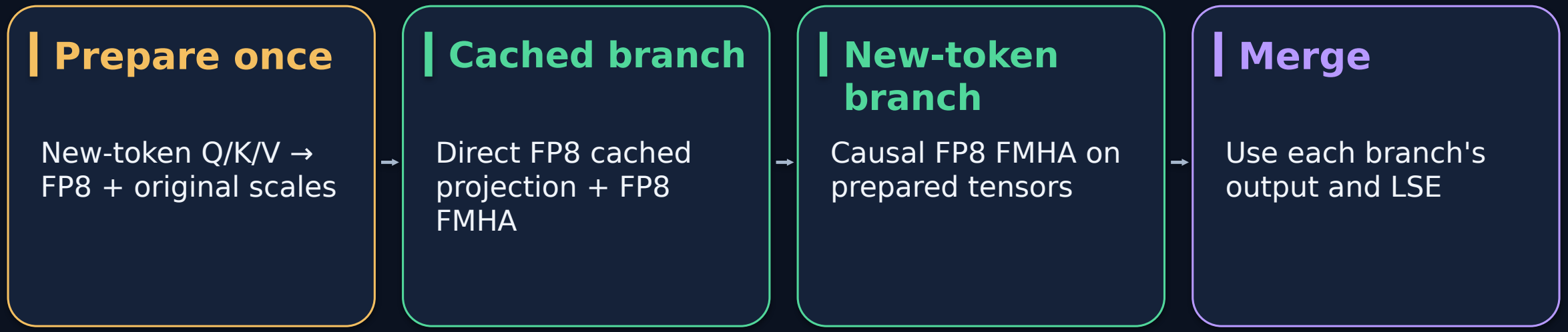
Logical sizes per token/rank

BF16 expanded K/V: 7,680 bytes; FP8: 3,840 bytes.

12 × (192 + 128) = 3,840 expanded values per token per TP rank

Stage 3: reuse prepared FP8 for new-token attention

The causal call consumes existing $Q_8/K_8/V_8$ with their original descales.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Two attention branches need two different masks

The new query chunk sees all old context, plus its allowed new-token prefix.



Old cached prefix

All its keys precede every new query

New-token block

Use a causal triangle within the chunk

Same query outputs

Merge by natural-log LSE

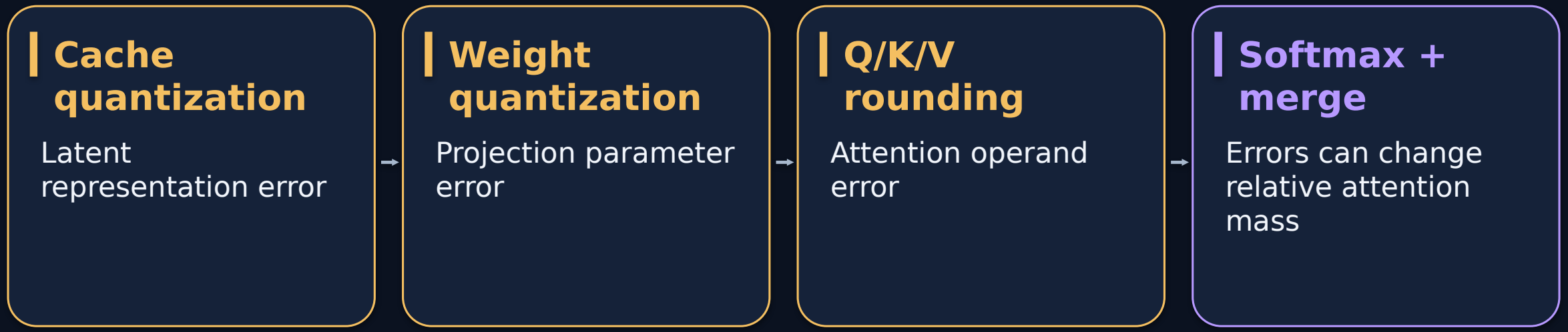
Scale discipline

Keep cached-output and new-token descales separate

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Quantization error enters at several points

A wider accumulator cannot restore information lost before multiplication.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The cached output scale is a policy to validate

Twice the new-token range is headroom—not a bound on all cached values.

New-token amax

Estimate current K/V ranges

Cached output scale

$2 \times \max(\text{new-token descale}, 10^{-6})$

Old cached outlier

May exceed that range

Validation

Measure clipping, output error and LSE error

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Validate three different questions separately

Arithmetic, quantization fidelity and model quality require different evidence.

Same FP8 operands

Does the kernel compute the intended quantized operation?

Original BF16 reference

How much error does the full quantized path introduce?

End-to-end model task

Does the executed implementation preserve task quality?

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Accuracy evidence is strong—but has a defined scope

The prototype passed full GSM8K; native-PR validation remains separate.

1,319

GSM8K questions

96.9674%

Strict: 1,279 correct

96.8916%

Flexible: 1,278 correct

8 workers

Optimized-path invocation verified

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Execution proof is part of correctness

A passing test is useful only if it exercised the optimized path.

Pin the source

Exact serving code and image

Verify operands

Dtypes, layouts and scales

Observe calls

Successful execution on all workers

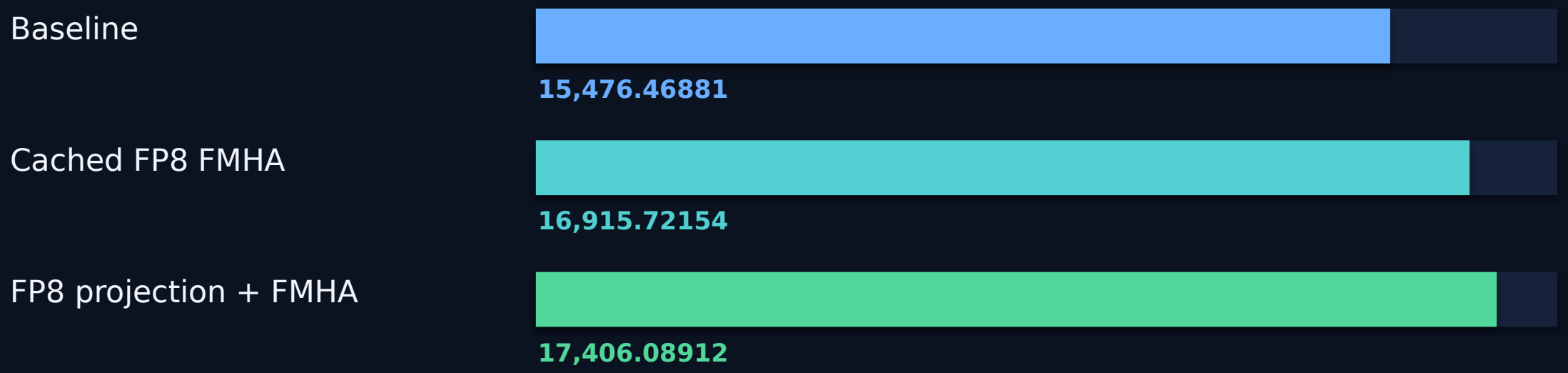
Separate phases

Warmup, capture, evaluation and replay

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

The three established performance stages

Total input + output tokens/s/GPU under the recorded full configurations.



+9.30% → +2.90% additional | +12.47% vs baseline

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Read throughput and latency together

A higher token rate can coexist with worse user-visible responsiveness.

TTFT

Time to first output token

ITL / TPOT

Spacing or average time per output token

Output tok/s

Actual generation rate

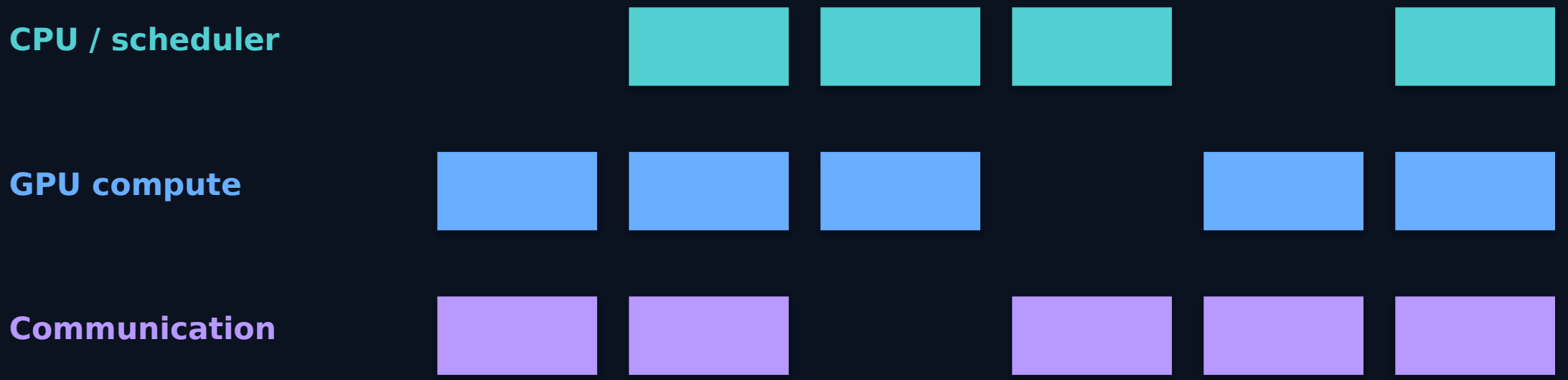
Total tok/s

Input + output, including workload reuse semantics

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Profile steady state before choosing the next kernel

Locate the critical path, then distinguish compute, memory and waiting.



CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Amdahl's law puts a ceiling on local speedups

The fastest kernel matters only for the fraction of time it can remove.

1. **Fraction f** — Share of end-to-end time affected
2. **Local speedup s** — Improvement to that portion
3. **Remaining work** — Unaffected time still has to execute

Overall speedup = 1 / [(1 − f) + f/s]

Choose the next experiment from the limiting resource

Change one hypothesis at a time and preserve comparable evidence.

Memory-bound

Remove an intermediate; improve reuse or dtype.

Launch / sync-bound

Fuse small work; inspect graph coverage and rank skew.

Batch / cache-bound

Sweep C56 + MTP, C64 + MTP, C96 without MTP.

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Trace one new token through the complete system

The architecture and serving machinery now form one connected picture.

Schedule

Choose tokens; resolve cache/state

Project

Create Q/K/V or KDA update inputs

Mix context

KDA recurrence or MLA attention

Transform

Latent MoE and shared branch

Mix depth

Attention Residuals

Generate

Logits, sampling and state commit

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Six questions that check understanding

Answer these before interpreting the next performance result.

1–2

Why cache K/V but usually not Q? Why does KDA need state recovery?

3–4

Why are TP8 + DCP8 still eight GPUs? Why does a legacy RoPE field not prove rotation?

5–6

What BF16 traffic does direct FP8 projection remove? Why is GSM8K not an LSE test?

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Tensor atlas: the cached-prefill path at TP8

Shapes shown per rank; token counts depend on the current packed call.

Hidden state X	[T, 7,168]
Compressed cache row	512 latent + 64 shared K values
kv_b_proj weight	[3,072, 512]
Query Q / key K	[Tq,12,192] / [Tk,12,192]
Value V / output O	[Tk,12,128] / [Tq,12,128]
LSE	[12,Tq] in this adapter

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Glossary: model concepts

Keep the mathematical role separate from the implementation.

Attention / FMHA

Weighted context retrieval / fused implementation

KDA / MLA

Recurrent delta attention / latent-cache attention

Latent / projection

Compressed feature space / learned matrix transform

MoE / router

Expert pool / token-to-expert selection

Residual / gate

Depth information path / learned modulation

LSE

Logarithm of unnormalized attention mass

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Glossary: execution concepts

Name the resource or mechanism before tuning it.

HBM / LDS

Device memory / workgroup-shared on-chip memory

CU / XCD / wave

Compute unit / GPU chiplet / 64-lane execution group

TP / DCP / EP

Tensor / decode-context / expert parallelism

NUMA / CCD

CPU-memory locality / CPU compute die

GEMM / MFMA

Matrix multiplication / matrix instruction family

Quant / descale

Round into compact format / restore numerical units

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES

Source map and further reading

Pinned model facts, implementation references and campaign evidence.

Model + architecture

Checkpoint, model card and AMD Kimi implementation

Algorithms + hardware

Attention, FlashAttention, Kimi Linear and ROCm specs

FP8 implementation

Campaign data flow, ATOM and AITER source

Evidence

Benchmark runs, GSM8K results and native PR status

CLICK TO BUILD THE DIAGRAM • REFERENCES AND EXPLANATION IN SPEAKER NOTES